

Kafka Producers

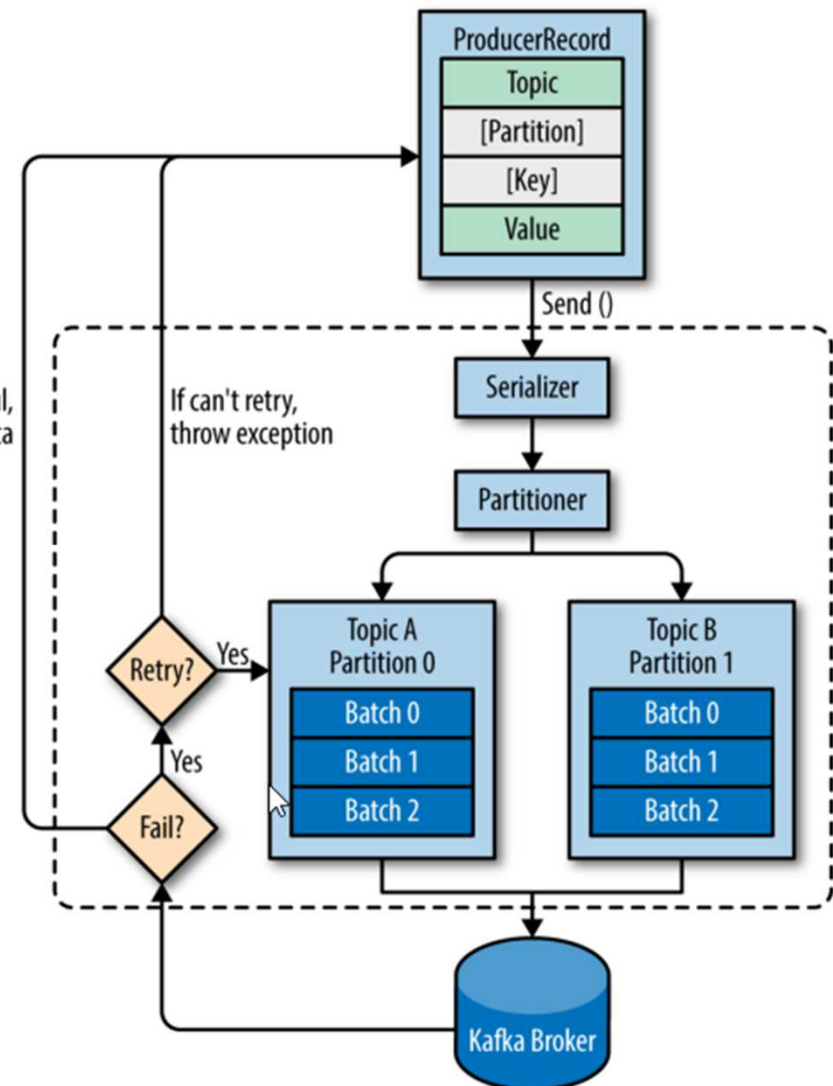
Overview of its design and components.

Objectives

- Kafka Producer Components
- Create a Producer Program
- Serializers
- Partitioners

Kafka Producer Components

- Produce message by creating 'ProducerRecord'
 - Include Topic, Value Objects as Byte Arrays.
 - Include optional Partition, Key
- Partitioner selects the partition when not specified in message, then adds record to batch of records to be sent to topic & partition.
 - Different thread is used to send the batch to topic & partition
- Broker receives message and sends back a response.
 - RecordMetadata object (topic, partition & offset) is returned when successfully written to Kafka
 - If failed error is returned to producer, can be retried



How to write Producer code

- A Kafka producer has three mandatory properties:
 - `bootstrap.servers` – List of `host:port` pairs of brokers that the producer will use to establish initial connection to the Kafka cluster.
 - `key.serializer` – Name of a class that will be used to serialize keys of records
 - `value.serializer` – Name of class that is used to serialize values of records
- Instantiate the producer
- Create a `ProducerRecord` with
 - Topic
 - Partition
 - Key and Value
- Start sending the messages in one of three ways
 - Fire and Forget – `producer.send(producerRecord)`
 - Synchronous send – `producer.send(producerRecord).get()`
 - Asynchronous send – `producer.send(producerRecord, new OurClassImplementingCallback());`
- Close this producer.

Producer Code

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

Producer<String, String> producer = new KafkaProducer<>(props);

for (int i = 0; i < 5; i++){
    ProducerRecord record = new ProducerRecord<String, String>
        ("test", Integer.toString(i), Integer.toString(i));
    producer.send(record);
}
producer.close();
```

Practice - Producer Program

- Set Classpath
 - `export CLASSPATH=./usr/local/kafka/libs/kafka-clients-0.9.0.1.jar:/usr/local/kafka/libs/slf4j-api-1.7.6.jar:/usr/local/kafka/libs/slf4j-log4j12-1.7.6.jar:/usr/local/kafka/libs/log4j-1.2.17.jar`
- Write the code
 - Take the code from 'OneWayProducerDemo - ReferenceCode'
 - Save code to `c:\labs\kafka\src` folder as "ProducerDemo.java"
- Compile
 - `cd /vagrant/src`
 - `javac ProducerDemo.java`
- Using a new command prompt, consume messages using shell script.
 - `/usr/local/kafka/bin/kafka-console-consumer.sh --zookeeper localhost:2181 --topic InvalidTopic`
Note: Can you fix error in the command above
- Run the program, and verify if message is consumed.
 - `java "ProducerDemo.java"`

Practice - Do it yourself

- Modify previous program `ProducerDemo.java` to send produced message
 - Key as 'myName' and value as 'Your Name'
 - Key as 'myJob' and value as 'Your Job'
 - Challenge – Answer the question?
 - Key as 'mySalary' and value as `your_salary` (Float)
Why can't we send employee id as integer value in our code?

```
record = new ProducerRecord<String, Integer>("test", "employeeId", 14948);  
producer.send(record);
```

Three ways of sending Data

- Sending a Message to Kafka
 - We have seen this earlier
- Sending a Message Synchronously
 - `producer.send(record).get();`
 - Here, we are using `Future.get()` to wait for a reply from Kafka.
 - This method will throw an exception if the record is not sent successfully to Kafka.
 - If there were no errors, we will get a `RecordMetadata` object that we can use to retrieve the offset the message was written to.
- Sending a Message Asynchronously
 - Create a class implementing `Callback` class method's `onCompletion` method
 - Send message Asynchronously using `Callback` back we wrote
 - `producer.send(record, new OurCallbackClass());`

Configuring Providers

<http://kafka.apache.org/documentation.html#producerconfigs>

- **acks** - how many partition replicas must receive the record before the producer can consider the write successful.
 - **acks=0**, producer will not wait for a reply from the broker before assuming the message was sent successfully
 - **acks=1**, producer will receive a success response from the broker the moment the leader replica received the message.
 - **acks=all**, producer will receive a success response from the broker once all in-sync replicas received the message.
- **buffer.memory** – buffer messages waiting to be sent to brokers.
- **compression.type** – which algorithm to use for compressing messages
- **retries** – how many times producer can retry when it receives error from server
- **batch.size** – When multiple records are sent to the same partition, the producer will batch them together. This parameter determines the size.
- **linger.ms** – amount of time to wait for additional messages before sending the current batch.
- **client.id** – Any string, used to identify which client sent the message
- **delivery.timeout.ms** - An upper bound on the time to report success or failure after a call to `send()` returns.

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("acks", "all");
props.put("delivery.timeout.ms", 30000);
props.put("batch.size", 16384);
props.put("linger.ms", 1);
props.put("buffer.memory", 33554432);
props.put("key.serializer", "org.apache.kafka.com
props.put("value.serializer", "org.apache.kafka.c
```

Serializers

- We've seen how to use the default String serializer.
- Kafka also includes serializers for integers and ByteArrays, but this does not cover most use cases. Hence, we look toward CustomSerializers.

- Create a class that represents your record.
- Implement 'org.apache.kafka.common.serialization.Serializer' interface to override 'serialize' method.
 - `serialize()` method returns a byte Array

```
public class Customer {  
    private int customerID;  
    private String customerName;  
    public Customer(int ID, String name) {  
        this.customerID = ID;  
        this.customerName = name;  
    }  
    public int getID() {  
        return customerID;  
    }  
    public String getName() {  
        return customerName;  
    }  
}
```

- Debugging compatibility issues between different versions of serializers and deserializers is fairly challenging, hence it is recommended to use existing serializers and deserializers such as JSON, Apache Avro, Thrift, or Protobuf.
 - Kafka Documentation for Datatypes –
<https://kafka.apache.org/10/documentation/streams/developer-guide/datatypes.html>
 - Avro documentation
<https://www.confluent.io/blog/avro-kafka-data/>

Partitions

Constructor and Description



`ProducerRecord(java.lang.String topic, java.lang.Integer partition, K key, V value)`

Creates a record to be sent to a specified topic and partition

- When partition is missing while creating `ProducerRecord`, Kafka will hash the key and use the result to map the message to a specific partition
 - Kafka uses its own hashing algorithm to address Java upgrade
- The mapping of keys to partitions is consistent only as long as the number of partitions in a topic does not change.
- When partitioning keys is important, the easiest solution is to create topics with sufficient partitions and never add partitions.
- Do it yourself
 - Modify earlier program to send name to partition 1 and job to partition 2 (Where does the program fail and why?)

Summary

- Kafka Producer Components
- Create a Producer Program
- Serializers
- Partitioners

Practice

- This practice teaches